

Player-cancelable internal cooldowns

An `internal_cd` classification gap lets players clear their own proc lockouts, and lets enemies clear them too

Summary. The aura kind `internal_cd` is missing from `DEBUFF_AURA_KINDS`, so every internal-cooldown lockout in the game classifies as a helpful buff. Helpful buffs are right-click cancelable and, when they carry a magic school, offensively dispellable. Because proc gates test only for the lockout aura's *presence*, removing it re-arms the proc immediately. This affects 28 lockout sites across 6 classes. The fix is a one-line change at the documented chokepoint.

1. What a player can do today

Take the hunter talent `hun_r14_guise_mastery` as the worked example. It grants a 6 second benefit when the hunter swaps Guise, gated behind a 20 second shared internal cooldown. The cooldown is stored as an aura:

```
// src/sim/combat/hunter_shared.ts - applyStateAura()
ctx.applyAura(entity, {
  id, name, kind: 'internal_cd', remaining: duration,
  duration, value, stacks, sourceId: entity.id, school: 'physical',
});

// the proc gate is a bare presence check
if (hunter.auras.some((aura) => aura.id === GUISE_MASTERY_ICD_ID)) return;
```

The lockout renders on the player's buff bar as an ordinary buff icon named "Guise Mastery". Right-clicking it removes it. The gate above then passes, and the hunter re-procs the talent on their next Guise swap. The 20 second cooldown is bypassed entirely, at will, with no cast, no resource cost, and no cooldown of its own.

This generalises: any proc whose lockout is an `internal_cd` aura and whose gate is a presence check can be re-armed the same way.

2. Root cause

The cancel decision resolves through a chain of four predicates:

```
// src/sim/aura_classify.ts
export const DEBUFF_AURA_KINDS = new Set<AuraKind>([ ... ]); // 'internal_cd' ABSENT

isDebuffAura(kind, value) -> DEBUFF_AURA_KINDS.has(kind)
                             || (kind.startsWith('buff_') && value < 0)
isPlayerRemovableAura(aura) -> !isUnbreakableControlAura(aura)
                               && aura.undispellable !== true

// src/sim/combat/aura_cancel.ts
isCancelableAura(a) -> isPlayerRemovableAura(a) && !isDebuffAura(a)
```

For a lockout aura: `internal_cd` is not in the debuff set and does not start with `buff_`, so `isDebuffAura` is **false**. `applyStateAura` sets neither `unbreakableControl` nor `undispellable`, so `isPlayerRemovableAura` is **true**. Therefore `isCancelableAura` is **true**.

That predicate is load-bearing in two places at once. `src/ui/auras_view.ts` sets `slot.cancelable = mode === 'buffs' && isCancelableAura(a)`, which puts the icon on the buff bar and attaches the right-click handler; `Sim.cancelAura` then calls `removeCancelableAura`, which re-tests the same

predicate before splicing. Both arms agree, so the cancel goes through cleanly.

This was a gap, not a decision. tests/aura_classify.test.ts pins a HARMFUL list and a HELPFUL list against isDebuffAura. internal_cd appears in neither, so no test ever asserted which side it belongs on. Note also that the two neighbouring lockouts sated and cauterize_fatigue are in the debuff set, each with a comment explaining that they are lockouts. internal_cd was simply missed.

3. Second manifestation: enemies can clear your lockout

The same root cause reaches the dispel path, and there it is worse because it crosses players:

```
// src/sim/aura_classify.ts - isDispellableAura()
if (aura.id === 'divine_ascension' || aura.permanent) return false;
if (!isPlayerRemovableAura(aura)) return false; // <- passes for internal_cd
if (aura.school === 'physical') return false; // <- only guard that catches it
const harmful = isDebuffAura(aura.kind, aura.value);
return offensive ? !harmful : harmful; // offensive -> TRUE
```

The school === 'physical' line is the only thing standing between lockouts and offensive dispel, and it is incidental rather than intentional. Most lockouts are not physical:

Module	School	Offensively dispellable
src/sim/combat/fire_mage.ts	fire	Yes
src/sim/combat/necromancy.ts	shadow	Yes
src/sim/combat/warlock_talents.ts	shadow	Yes
src/sim/combat/shaman_thundercall.ts	nature	Yes
src/sim/combat/paladin_talents.ts	holy	Yes
src/sim/combat/hunter_shared.ts	physical	No (incidental)

So an offensive dispel or Spellsteal aimed at an enemy currently **removes their proc lockout**, which helps the target rather than hurting them. Please also check what Spellsteal does when it copies an internal_cd aura onto the caster, since that would hand the stealer a lockout they never earned.

4. Blast radius

28 kind: 'internal_cd' creation sites across 14 modules and 6 classes. None of them set undispellable.

Area	Modules
Hunter	combat/hunter_shared.ts, combat/hunter_fieldcraft.ts, combat/hunter_packlord.ts
Shaman	combat/shaman_talents.ts, combat/shaman_thundercall.ts, combat/shaman_warspirit.ts
Warlock	combat/warlock_talents.ts, combat/necromancy.ts
Paladin	combat/paladin_talents.ts, combat/paladin_valkyrs_calling.ts, paladin_devotion.ts
Mage	combat/fire_mage.ts, combat/convergence.ts

Area	Modules
Shared	combat/casting_lifecycle.ts

5. Recommended fix

Make `isPlayerRemovableAura` reject `internal_cd`. That function is already documented in `src/sim/aura_classify.ts` as "the one rule for may a player counter take this aura off at all", and every removal path a player can drive is routed through it: the dispel executor, the `requiresDispellable` cast gate, the `cleanseSelf` executor, and the right-click cancel. A lockout is precisely an aura that no counter may shed, so this is the semantically correct home and it closes all four paths in one edit.

```
// src/sim/aura_classify.ts

// A lockout the sim owns: internal cooldowns, proc windows, and rolling-window
// accumulators gate their own re-arm on the aura's PRESENCE, so letting a player
// take one off (right-click cancel, cleanse, or an enemy's offensive dispel) hands
// them a free cooldown reset. Not a debuff: it still renders on the buff bar so the
// player can watch it tick down.
export function isPlayerRemovableAura(
  aura: Pick<Aura, 'kind' | 'unbreakableControl' | 'undispellable'>,
): boolean {
  if (aura.kind === 'internal_cd') return false; // <-- ADD THIS
  return !isUnbreakableControlAura(aura) && aura.undispellable !== true;
}
```

Why this shape and not the obvious alternatives

- **Do not** add `internal_cd` to `DEBUFF_AURA_KINDS`. That would fix cancelability, but it also moves every lockout from the buff bar to the debuff bar and restyles it red. Several lockouts are deliberately player-facing; `combat/casting_lifecycle.ts` says outright that its accumulator "rides an `internal_cd` aura the player can watch tick down". That is a visible UX regression for a correctness bug.
- **Do not** set `undispellable: true` at each creation site. It works, but it is 28 edits, it will be forgotten at the 29th, and it says nothing about why.
- **Do not** harden the proc gates one by one (checking a timestamp instead of aura presence). That is a much larger diff across 6 classes and leaves the dispel hole open.

One consequence worth calling out in review: the fix makes the existing `divine_ascension` special case in `isDispellableAura` redundant, since that aura is `internal_cd`. Leave it in place (it is also keyed on permanent) but do not be surprised that its test still passes for a new reason.

6. Tests to add

Per `CLAUDE.md`, a sim change gets a test, and the bug should be reproduced first. Suggested coverage:

File	Assertion
<code>tests/aura_classify.test.ts</code>	Add <code>internal_cd</code> to a pinned lockout list: <code>isPlayerRemovableAura</code> is false, and <code>isDispellableAura</code> is false for BOTH polarities at every school (fire, shadow, nature, holy, physical). This is the regression pin that stops the kind drifting back.

File	Assertion
tests/aura_classify.test.ts	Keep <code>isDebuffAura('internal_cd', 1) === false</code> , so the buff-bar rendering contract is pinned as deliberate rather than left unstated.
New: tests/aura_cancel.test.ts	<code>removeCancelableAura</code> returns null for an <code>internal_cd</code> aura and leaves the array intact; it still removes an ordinary <code>buff_ap</code> .
tests/hunter_lifecycle.test.ts	End to end on the real path: <code>proc Guise Mastery</code> , call <code>sim.cancelAura(GUISE_MASTERY_ICD_ID, pid)</code> , swap Guise again inside 20 seconds, assert NO second proc. Write it failing first.
tests/world_api_parity.test.ts	No change expected. Confirm anyway, since <code>cancel_aura</code> is a pinned <code>IWorld</code> member.

7. Verification before you push

- Check that the flag-drop path still works. `Sim.cancelAura` intercepts the carried-flag aura through `ctx.bgCancelFlagAura` before it reaches `removeCancelableAura`, and that aura uses its own `flag_carried` kind, so it should be unaffected. Confirm with the battleground tests rather than by reading.
- Confirm no legitimate gameplay cancels an `internal_cd`. A repo-wide grep for `cancelAura` call sites is cheap and worth doing.
- Run `node scripts/gate_select.mjs` before calling it done.

Version note. The code quoted here was read from `release/v0.37.0`, which is the newest release branch on the remote. Production currently runs build **0.39.3**. Re-confirm each quoted site against the current release branch before writing the patch; the shape of the bug is very unlikely to have changed, but line-level details may have.

8. Why this is a code fix and not a moderation matter

This was found while investigating whether hunters had been cancelling Guise Mastery in ranked play. Two findings are relevant to how the fix gets framed.

- **Cancellation is not recorded anywhere.** `Sim.cancelAura` emits a bare `{ type: 'aura', targetId, name, gained: false }`, identical to a fade from expiry, dispel, death, or overwrite. The parse recorder enriches only gained events, and the `cancel_aura` handler in `server/game.ts` writes no audit log. If you want this observable in future, that is a separate change: widen the fade event with a removal reason and carry it into `server/parse/contract.ts`.
- **No exploitation was detected.** Because the proc gate blocks re-application while the lockout is up, each lockout should run its full 400 ticks, capping procs at 3 per minute. Across the sampled battlegrounds the only player running the talent averaged one proc per 44 to 57 seconds, with mean lockout duration at 369 to 421 ticks. That is consistent with normal play and shows no cancellation.

The lockout is presented to the player as a right-clickable buff icon by the game's own HUD. Anyone who clicked it used an affordance the client offered them. Fix the classification; there is nothing here to enforce against.